Topic 4 – Object methods, "this"

1. Method يعني إيه؟

لما نحط Function جوه Object كـ property، بنسميها Method:

```
const user = {
  name: "Yousef",

  sayHi() {
    console.log("Hello");
  }
};

user.sayHi();
```

الأفضل في الشغل

استخدم Method Shorthand:

```
sayHi() {
  console.log("Hello");
}
```

بدل:

```
sayHi: function () {
  console.log("Hello");
}
```

الاتنين شغالين، لكن الـ shorthand هو الأسلوب الأفضل والأكثر شيوعًا.

2. this في الـ Methods

أهم قاعدة في الموضوع كله:

this بتتحدد حسب طريقة استدعاء الـ Function، مش حسب مكان كتابة الـ Function.

لما تعمل:

```
user.sayHi();
```

يبقى:

```
this === user
```

مثال:

```
const user = {
  name: "Yousef",

  sayHi() {
```

```
    console.log(this.name);
  }
};

user.sayHi();
```

الناتج:

```
Yousef
```

لأن:

```
this.name
```

هي:

```
user.name
```

3. ليه نستخدم this بدل اسم الـ Object؟

بدل:

```
const user = {
  name: "Yousef",

  sayHi() {
    console.log(user.name);
  }
};
```

الأفضل:

```
const user = {
  name: "Yousef",

  sayHi() {
    console.log(this.name);
  }
};
```

لأن this بتخلي الـ Method reusable.

مثلاً نفس الـ Function:

```
function sayHi() {
  console.log(this.name);
}

const user = {
  name: "Yousef",
  sayHi
```

```
};

const admin = {
  name: "Admin",
  sayHi
};
```

هنا:

```
user.sayHi();  // Yousef
admin.sayHi(); // Admin
```

نفس الـ Function، لكن this بتتغير حسب مين استدعاها.

4. this مش ثابتة

دي نقطة Interview مهمة.

```
function sayHi() {
  console.log(this.name);
}
```

الـ Function نفسها مش بتقول:

أنا تبع user.

لأ.

اللي بيحدد this هو call site.

```
user.sayHi();
```

→ this = user

```
admin.sayHi();
```

→ this = admin

5. Dot و Bracket

الاتنين بيحددوا this للـ Object:

```
user.sayHi();
```

و:

```
user["sayHi"]();
```

الاتنين:

this === user
6. Function عادية بدون Object

لو عندك:

```
function sayHi() {
  console.log(this);
}

sayHi();
```

في strict mode:

```
this === undefined
```

ده كان سبب الـ Task الأول.

7. أهم فرق: Normal Function vs Arrow Function
Normal Function

عندها this خاص بيها، وبيتحدد حسب طريقة استدعائها.

Arrow Function

معندهاش this خاص بيها.

بتاخد this من الـ surrounding scope.

مثال:

```
const user = {
  name: "Yousef",

  sayHi() {
    const arrow = () => {
      console.log(this.name);
    };

    arrow();
  }
};

user.sayHi();
```

النتيجة:

```
Yousef
```

لأن:

```
user.sayHi()
      ↓
this = user
      ↓
arrow ليس لها this
      ↓
تستخدم this الخاصة بـ sayHi
      ↓
this = user
```

احفظها:

```
Normal Function → own this

Arrow Function → no own this → inherits outer this
```

8. خطأ مشهور جدًا ❌

متحطش Arrow Function كـ Method لو محتاج this بتاع الـ Object:

```
const user = {
  name: "Yousef",

  sayHi: () => {
    console.log(this.name);
  }
};
```

إن this تكون user. ده مش صح لو هدفك

الأفضل:

```
const user = {
  name: "Yousef",

  sayHi() {
    console.log(this.name);
  }
};
```

9. استخدام عملي — Method داخل Arrow مع this

وده من أهم الاستخدامات في Front-End:

```
const user = {
  name: "Yousef",

  sayHi() {
    setTimeout(() => {
      console.log(this.name);
    }, 1000);
  }
```

```
};

user.sayHi();
```

هنا مفيدة لأنها تحافظ على Arrow الـ this الخاصة بالـ Method.

10. Task 1 – this داخل Object Literal

الكود:

```
function makeUser() {
  return {
    name: "John",
    ref: this
  };
}

let user = makeUser();

alert(user.ref.name);
```

makeUser() اتعملها استدعاء عادي:

```
makeUser();
```

إذن في strict mode:

```
this === undefined
```

فيبقى تقريبًا:

```
{
  name: "John",
  ref: undefined
}
```

وبعدين:

```
user.ref.name
```

تبقى:

```
undefined.name
```

وده يعمل:

```
TypeError
```

مهم:

مش undefined فقط.

الـ ref نفسها undefined، لكن محاولة الوصول إلى .name على undefined هي اللي تعمل TypeError.

11. Task 1 – الحالة العكسية
```
function makeUser() {
  return {
    name: "John",

    ref() {
      return this;
    }
  };
}

let user = makeUser();

alert(user.ref().name);
```

هنا:

```
user.ref()
```

دي Method Call.

إذن:

```
this === user
```

والـ:

```
return this;
```

ترجع الـ user نفسه.

فتبقى:

```
user.ref().name

→ "John"
```

القاعدة:
```
obj.method()

→ this = obj
```

12. Task 2 – Calculator
```
let calculator = {
  sum() {
    return this.a + this.b;
  },
```

```
  mul() {
    return this.a * this.b;
  },

  read() {
    this.a = +prompt("a?", 0);
    this.b = +prompt("b?", 0);
  }
};
```

لو دخلنا:

```
5
3
```

بعد:

```
calculator.read();
```

يبقى:

```
calculator.a = 5;
calculator.b = 3;
```

ثم:

```
calculator.sum();
```

→ 8

و:

```
calculator.mul();
```

→ 15

خلي بالك

this 3 ولا 5 هنا مش.

this هي:

```
calculator
```

هما 3و5 والـ values موجودة في:

```
this.a
this.b
```

13. Task 3 – Chaining

المشكلة:

```
up() {
  this.step++;
}
```

Function بدون return:

```
ladder.up()
```

ترجع:

```
undefined
```

فده:

```
ladder.up().up();
```

يبقى:

```
ladder.up()
     ↓
undefined
     ↓
undefined.up()
     ↓
❌ TypeError
```

الحل:

```
up() {
  this.step++;
  return this;
}
```

ليه؟

لأن:

```
this === ladder
```

فـ:

```
return this;
```

يعني:

```
return ladder;
```

وبالتالي:

```
ladder.up().up()
```

تشتغل.

14. return this vs return this.step

دي فهمتها كويس جدًا 👌

```
return this.step;
```

ترجع القيمة:

```
1
```

لكن:

```
return this;
```

ترجع الـ Object نفسه:

```
ladder
```

وعشان الـ chaining محتاج نكمل Method بعد Method:

```
ladder.up().up().down()
```

لازم نرجع الـ Object، مش الـ number.

احفظها:
```
return this.step;
```

= هات القيمة

```
return this;
```

= رجّعلي الـ Object عشان أكمل

15. Final Chaining Example
```
let ladder = {
  step: 0,

  up() {
    this.step++;
    return this;
  },

  down() {
    this.step--;
```

```
    return this;
  },

  showStep() {
    alert(this.step);
    return this;
  }
};
```

لما نعمل:

```
ladder.up().up().down().showStep();
```

الحساب:

```
0
↓ up
1
↓ up
2
↓ down
1
↓ showStep
1
```

إذن:

```
1
🔥 أهم Tricks في Topic
Trick 1
```

لما تشوف:

```
obj.method()
```

فكر فورًا:

```
this === obj
Trick 2
```

لما تشوف:

```
function () {}
```

اسأل:

مين استدعاها؟

لأن ده غالبًا هيحدد this.

Trick 3

لما تشوف:

```
() => {}
```

متدورش على this جوا الـ Arrow نفسها.

اسأل:

منين؟ this هي أخدت

Trick 4
this

مش معناها:

الـ Object اللي الـ Function مكتوبة جواه.

لكن معناها:

القيمة اللي الـ Function اتستدعت من خلالها.

Trick 5

لو Function مفيهاش return:

```
function test() {}
```

فهي ترجع:

```
undefined
```

لكن ده مش خطأ في حد ذاته.

الخطأ يحصل لما تحتاج القيمة الراجعة وتستخدمها:

```
test().something(); // ✕
```
Trick 6

في Chaining:

```
return this;
```

مش عشان نرجع قيمة الـ object.

لكن عشان نرجع الـ object نفسه ونكمل عليه Methods.

أشهر الأخطاء ✕
الصح الخطأ

this هي الـ object اللي اتكتبت فيه Function this تتحدد حسب الاستدعاء
Arrow عندها this خاص Arrow لا تملك this خاص
لازم كل Function تعمل return فقط لو محتاج القيمة الراجعة
return this.step يعمل chaining return this هو المطلوب
console.log() يرجع القيمة console.log() يعرض، return يرجع
undefined = TypeError undefined قيمة؛ التعامل معها كـ object قد يسبب TypeError
استخدامه في الشغل 💼

هتقابل نفس الفكرة في:

Object methods
Classes
React/JavaScript callbacks
Event handlers
setTimeout
APIs والمكتبات

Method chaining زي:

```
arr.filter(...).map(...).slice(...)
```

this وفهم مهم جدًا قبل ما تدخل أعمق في Classes / Prototypes / OOP.

🎯 Interview Questions
Q1: What is this?

Answer:

this refers to the object/context associated with the current function call. Its value depends on how the function is called.

Q2: What is the difference between normal and arrow functions regarding this?

Answer:

A normal function has its own this determined by how it is called, while an arrow function doesn't have its own this; it inherits it from the surrounding scope.

Q3:

```
const user = {
  name: "Yousef",

  sayHi: () => {
    console.log(this.name);
  }
};
```

هل this === user؟

No ❌

الـ Arrow لا تأخذ this من user.

Q4:
```
const user = {
  name: "Yousef",

  sayHi() {
    console.log(this.name);
  }
};

user.sayHi();
```

this تساوي إيه؟

user ☑

Q5: Why do we use return this in chaining?

Answer:

To return the same object so another method can be called immediately on it.

🤓 Final Cheat Sheet

احفظ دول بس لو عايز تختصر الموضوع:

```
obj.method()
→ this = obj

normal function
→ this depends on invocation

arrow function
→ no own this
→ inherits outer this

no return
→ undefined

return this
→ return the object itself

return this.step
→ return the step value

console.log()
→ displays
```

return
→ sends value back
☑ Topic 4 Status